

經管資料分析

Day 2 Python、pandas、SQL

與測試工具書

以 management-review-agent

專案為例。重點是程式邏輯、常用寫法與可追溯分析，不只是背 API。

使用方法

每一節都回答：這段程式解決什麼問題、常用語法代表什麼、最容易犯什麼錯、何時應該寫測試。遇到類似情境時，直接回查對應章節。

1. 一條可信的分析流程

步驟	目的	本專案函式
讀取	CSV 轉成日期與數字型別	load_kpi_data
驗證	阻止壞資料進入分析	validate_kpi_data
計算	新增 KPI 與零分母警告	calculate_kpis
比較	預算、月增、年增、趨勢、排名	compare_*、rank_*
追查	把異常連回事件線索	get_incident_details
驗證結果	測試與 SQL 對照	pytest、sql_analysis.py

核心原則：事實、規則判定、原因假設要分開。KPI 數字是事實。threshold 是規則。incident 是待查證線索，不是已確認根因。

分析前先確認：一列資料代表什麼？哪些欄位可以加總？比較期間是否真的存在？資料是否已關帳？分母是否為 0？

2. 函式說明就是資料契約

好的 docstring 不只是註解，它告訴其他人這個函式能接受什麼、會產出什麼，以及何時不能下結論。

```
def function_name(df: pd.DataFrame, period: str) -> pd.DataFrame:
    """
    功能：解決哪個決策問題。
    參數：df 需要哪些欄位？period 格式是什麼？
    回傳：每列代表什麼？有哪些欄位？
    例外/警告：缺資料、零分母、preliminary 如何處理？
    """
```

寫程式前，先寫輸入欄位、輸出欄位和 warning 規則。這會讓你的函式更容易拆分、使用與測試。

3. pandas：選列與條件

寫法	意思	例子
df.loc[mask]	依布林條件取列	df.loc[df['plant'].eq('Tainan')]
.eq(x)、.ne(x)	等於、不等於	revenue.ne(0)
.gt(x)、.ge(x)	大於、大於等於	z_score.abs().ge(2)
.lt(x)、.le(x)	小於、小於等於	variance.le(-10)
.isna()、.notna()	缺值、非缺值	baseline.notna()
.isin(values)	是否在指定清單	period.isin([current, previous])
.between(a, b)	介於兩個邊界	period.between(start, end)
~mask	反轉 True/False	~preliminary_data

```
target = pd.to_datetime("2026-09", format="%Y-%m")
mask = (
    df["period"].eq(target)
    & df["data_status"].eq("closed")
)
```

```
& df["revenue"].gt(0)
)
closed_rows = df.loc[mask].copy()
```

重要：pandas 條件請用 `&`、`|`、`~`，不要用 `and`、`or`、`not`。每個條件都加括號，避免優先順序錯誤。

4. 鏈式寫法與 groupby

鏈式寫法把資料流程接起來。讀法是：先 `groupby`，再 `agg`，最後 `sort_values`。超過 3 到 5 段，或需要逐步檢查時，請拆成中間變數。

```
plant_monthly = (
    scope.groupby(["period", "plant"], as_index=False)
    .agg(
        revenue=("revenue", "sum"),
        cost=("cost", "sum"),
        source_rows=("department", "size"),
    )
    .sort_values(["plant", "period"])
)
```

Named aggregation 的格式是 新欄位=(原欄位, 聚合方式)。size 計算群組列數，適合 `source_rows`。count 則忽略缺值，兩者不同。

先聚合、再比率：成本率必須是總成本 / 總營收。不要平均部門成本率，因為部門營收權重不同。

```
result = df.groupby("plant", as_index=False).agg(
    revenue=("revenue", "sum"),
    cost=("cost", "sum"),
)
result["cost_rate"] = result["cost"].div(result["revenue"]).mul(100)
```

5. 日期與比較基準

```
target = pd.to_datetime(period, format="%Y-%m")
previous_month = target - pd.DateOffset(months=1)
previous_year = target - pd.DateOffset(years=1)
scope = df.loc[df["period"].isin([target, previous_month])].copy()
```

缺少 2026-08 時，不能把 2026-07 當成 2026-09 的上個月。比較基準必須是明確日期。缺資料時保留列並標示 `missing_previous_period`。

6. 零分母、缺值與 preliminary

```
result["cost_rate"] = (
    result["cost"]
    .div(result["revenue"])
    .mul(100)
    .where(result["revenue"].ne(0))
)
result.loc[result["revenue"].eq(0), "warning"] = "zero_denominator:cost_rate"
```

`.where(condition)` 會保留 True 的值，False 改為 NaN。這比 infinity 安全。preliminary 可以保留數字，但 `comparison_allowed` 必須是 False，不能放入正式排名。

情況	數字	結論
分母為 0	NaN	zero_denominator warning
缺去年同期	基準值 NaN	不可比較
preliminary	可顯示	不做正式排名或結論
粒度不同	先各自聚合	不可直接 merge 後加總

7. merge、趨勢、排名

```
result = current.merge(
    previous,
    on="plant",
    how="left",
    validate="one_to_one",
)
```

merge 是資料庫 JOIN。`how='left'` 保留本月所有廠別。`validate='one_to_one'` 是防呆：主鍵重複時直接報錯，避免多對多 JOIN 讓金額被重複放大。

```
eligible = result["comparison_allowed"]
result.loc[eligible, "metric_rank"] = (
    result.loc[eligible, "metric_value"]
    .rank(method="min", ascending=False)
    .astype("Int64")
)
```

排名前先排除不可比較資料。`pd.NA` 代表『不適用或未知』，不要用 0 代替，否則讀報表的人會誤以為排名最後。

8. SQL：CTE、CASE、NULLIF、LAG

SQL 與 Python 必須使用相同口徑。CTE 用來拆解中間步驟。CASE 是條件判斷。NULLIF(x, 0) 避免除以 0。

```
WITH monthly_with_previous AS (
    SELECT
        period, plant, revenue, cost,
        LAG(period) OVER (PARTITION BY plant ORDER BY period) AS previous_period,
        LAG(cost) OVER (PARTITION BY plant ORDER BY period) AS previous_cost
    FROM plant_month
)
SELECT * FROM monthly_with_previous
WHERE period = :target_period;
```

LAG 的陷阱：LAG 是排序後上一列，不一定是真正上個月。所以仍要確認 `previous_period` 是否等於目標月減一個月。

9. z-score 與事件追查

```
z_score = (value - values.mean()) / values.std(ddof=0)
statistical_anomaly = abs(z_score) >= threshold
```

z-score 只是輔助訊號，正式異常仍用業務門檻。樣本太少或標準差為 0 時，統計結果不穩定。事件明細是待查證線索，不能直接寫成根因。

10. pytest：把規則變成可重跑證據

```
def test_preliminary_data_cannot_be_compared():
    row = result.loc[result["plant"].eq("Taichung")].iloc[0]
    assert not row["comparison_allowed"]
    assert row["warning"] == "preliminary_data"
```

```
def test_validation_rejects_negative_value():
    corrupted.loc[0, "cost"] = -1
    with pytest.raises(ValueError, match="不可為負數"):
        validate_kpi_data(corrupted)
```

assert 後的表達式為 True 才通過。assert not x 表示 x 應為 False。pytest.raises 用來測『程式必須拒絕壞資料』。

測試類型	你應驗證什麼
正常案例	數字、欄位、排序是否正確
資料不足	missing warning、comparison_allowed 是否正確
零分母	NaN 而非 infinity
資料品質	缺值、重複主鍵、負數、不合法 status 是否被拒絕
SQL 核對	核心 KPI 與 Python 是否相同，浮點數用 pytest.approx

11. 除錯清單與最後口訣

問題	先看哪裡
ModuleNotFoundError	在專案根目錄使用 python -m pytest -q
比率怪怪的	是否先聚合分子與分母
merge 後金額變大	key 是否唯一，是否加 validate
排名怪怪的	preliminary 是否排除，ascending 是否正確
NaN 太多	缺資料還是零分母，warning 是什麼
SQL 與 Python 不同	期間、資料狀態、粒度、四捨五入

先驗證，再計算；先聚合，再比率；先確認期間，再比較；先標示限制，再下結論；先寫測試，再相信結果。